

7. Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

Input:

```
GNU nano 8.7.1 practical7.c
#include <stdio.h>
#include <stdlib.h>

int global = 100;
static int s = 50;

int main()
{
    int stack = 10;
    int *heap = (int*)malloc(sizeof(int));

    printf("Code Segment : %p\n", main);
    printf("Global Segment : %p\n", &global);
    printf("Static Segment : %p\n", &s);
    printf("Heap Segment : %p\n", heap);
    printf("Stack Segment : %p\n", &stack);

    free(heap);
    return 0;
}
```

Output:

```
sushmitha@LAPTOP-B907B06L:~$ nano practical7.c
sushmitha@LAPTOP-B907B06L:~$ gcc practical7.c -o practical7
sushmitha@LAPTOP-B907B06L:~$ ./practical7
Code Segment : 0x5bc627fe71a9
Global Segment : 0x5bc627fea010
Static Segment : 0x5bc627fea014
Heap Segment : 0x5bc64eb93010
Stack Segment : 0x7fffcfcfd187c
sushmitha@LAPTOP-B907B06L:~$ |
```

8a. Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

Input:

```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *a, *b;

    a = (int*)malloc(5 * sizeof(int));

    b = (int*)calloc(5, sizeof(int));

    a = (int*)realloc(a, 10 * sizeof(int));

    free(a);
    free(b);

    return 0;
}

```

output:

```

sushmitha@LAPTOP-B907B06L:~$ nano practical8a.c
sushmitha@LAPTOP-B907B06L:~$ gcc practical8a.c -o practical8a
sushmitha@LAPTOP-B907B06L:~$ ./practical8a
sushmitha@LAPTOP-B907B06L:~$ sudo apt update
[sudo: authenticate] Password:
Hit:1 http://archive.ubuntu.com/ubuntu resolute InRelease
Get:2 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Get:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:4 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [543 kB]
Get:5 http://archive.ubuntu.com/ubuntu resolute-backports InRelease [137 kB]
Get:6 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages [679 kB]
Get:7 http://security.ubuntu.com/ubuntu resolute-security/main Translation-en [127 kB]
Get:8 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Components [46.7 kB]
Get:9 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Packages [187 kB]
Get:10 http://security.ubuntu.com/ubuntu resolute-security/universe Translation-en [59.3 kB]
Get:11 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Components [43.4 kB]
Get:12 http://security.ubuntu.com/ubuntu resolute-security/restricted amd64 Packages [443 kB]
Get:13 http://security.ubuntu.com/ubuntu resolute-security/restricted Translation-en [87.8 kB]
Get:14 http://archive.ubuntu.com/ubuntu resolute-updates/main Translation-en [162 kB]
Get:15 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Components [98.0 kB]
Get:16 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Packages [308 kB]
Get:17 http://archive.ubuntu.com/ubuntu resolute-updates/universe Translation-en [98.9 kB]
Get:18 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Components [190 kB]
Get:19 http://archive.ubuntu.com/ubuntu resolute-updates/restricted amd64 Packages [457 kB]
Get:20 http://archive.ubuntu.com/ubuntu resolute-updates/restricted Translation-en [90.8 kB]
Get:21 http://archive.ubuntu.com/ubuntu resolute-backports/universe amd64 Components [1056 B]
Fetched 4033 kB in 3s (1161 kB/s)
14 packages can be upgraded. Run 'apt list --upgradable' to see them.
sushmitha@LAPTOP-B907B06L:~$ sudo apt install valgrind
valgrind is already the newest version (1:3.26.0-0ubuntu1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 14
sushmitha@LAPTOP-B907B06L:~$ valgrind --leak-check=full ./memory
valgrind: ./memory: No such file or directory
sushmitha@LAPTOP-B907B06L:~$ echo $?
127

```

8b. Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

Input:

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>

int main()
{
    int x = 100;

    if (fork() == 0)
    {
        printf("Child Before=%d\n", x);
        x = 500;
        printf("Child After=%d\n", x);
    }
    else
    {
        sleep(2);
        printf("Parent=%d\n", x);
    }

    return 0;
}
```

output:

```
sushmitha@LAPTOP-B907B06L:~$ nano practical8b.c
sushmitha@LAPTOP-B907B06L:~$ nano practical8b.c
sushmitha@LAPTOP-B907B06L:~$ gcc practical8b.c -o practical8b
sushmitha@LAPTOP-B907B06L:~$ ./practical8b
Child Before=100
Child After=500
Parent=100
sushmitha@LAPTOP-B907B06L:~$ |
```